Comparative study of symbolic execution tools applied to vulnerability detection

André Cardoso
School of Technology
Polytechnic Institute of Cávado and Ave
Barcelos, Portugal
a18848@alunos.ipca.pt

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert**

I. INTRODUCTION

In the digital era, it is evident that most individuals interact with devices running software that may be susceptible to exploitation. A fundamental principle in cybersecurity is that all software contains vulnerabilities; therefore, its security posture depends primarily on the difficulty of detection. The risk of attack correlates with the diligence of security teams in remediation efforts and the potential incentives available to attackers.

A. Motivation and Background

Considering this, software organizations must take action in defending their codebases in the most efficient manner when it comes to resources and time. Manual validations are based in human interaction, which is known to be very prone to errors, making it not cost-efficient. Also, as an application evolves in complexity, manual validation is not viable when it comes to the resources required, but also time as this is an operation that requires celerity. This procedure is often described as AST (Application Security Testing).

As such, many tools to find vulnerabilities in an automated fashion have been developed and distributed, each with their own set of pros and cons. The tools in question can be categorized as static or dynamic, and each on their own use different technics to accomplish different results. SAST (Static Application Security Testing) ultimately relies on some variety of pattern matching, data-flow analysis, and symbolic execution [1], while DAST (Dynamic Application Security Testing) excels at mimicking realistic attacks and threats [2].

B. Objectives

The objective is a comparison of two symbolic execution tools and see how they compare in terms of performance and their ability to detect vulnerabilities. To achieve this comparison, the tools must run on projects that are preferably coded in the same language, but if these tools do not support the same set of languages, the projects should have similar architecture or business logic/intent (e.g.: online stores, accounting services, ...). These tools will also be compared on how they can be tuned in order to achieve better results.

For a better overview of how the comparison will be performed, here follows a set of questions and metrics used to measure the tools:

- Time to test a project
- CPU usage while testing a project
- Memory usage while testing a project
- Analysis of vulnerabilities detection (Youden's Index)
- Complexity of the vulnerabilities detected
- Ability to find vulnerabilities out-of-the-box
- Ability to find vulnerabilities with proper manipulation
- Effort of tuning the tool

In sum, the expected output is a clear comparison of tools that can assist developers and software companies in keeping code secure proactively and explain what differs between them and why one should pick one over the other(s). These is how we intend to answer the following research questions:

- **Which tool is more effective at finding vulnerabilities?**
- **Which tool is more efficient in terms of performance?**

II. LITERATURE REVIEW

The security of an application showed its first impacts around the beginning of the century with the increasing popularity of the WWW (World Wide Web) and the enablement of user-fuelled websites (like social media). This rapid evolution paired with the common lack of knowledge of the users but also of the developers caused the birth of Denial of Service and Cross-Site Scripting attacks among others [3].

A. Application Security Testing

AST (Application Security Testing) includes every procedure that evaluates software security, these methods may include manual or automatic testing; manual testing may perform in the form of penetration tests, but automatic testing has 2 common variants, static (SAST) and dynamic (DAST).

Static Application Security Testing is a white-box approach to software security where we have access to source code and through analysis a developer can attempt to find vulnerabilities in it; its practice is also commonly described as finding code errors.

Dynamic Application Security Testing is on the other hand also known as the black-box approach – it has no knowledge of how a software is coded or designed. It only knows how to interact with the software and attempts to do so in a manner that it can find malfunctions and failures in the program execution; its results are the inputs used to break the application flow.

1) SAST vs. DAST: After looking at the research performed by [4], a summarized version of the comparison between the two methods can be described as seen in Table I.

SAST	DAST
White-box testing	Black-box testing
Test from the inside out	Test from the outside in
Test internal behaviour	Test external expectations
Validate code quality	Validate system functionality
High granularity	Low granularity
Full knowledge of application structure	No knowledge of application structure

TABLE I: Summarized Theoretical Comparison of SAST & DAST

This theoretical comparison is presented with a more practical one, where [4] tests, analyses, and studies different scans with different tools for both methods. The metrics used are the standard True Positive, True Negative, False Positive, False Negative (TP, TN, FP, FN) and are required to calculate the True Positive Rate (TPR) and False Positive Rate (FPR) which happen to be the most relevant rates that contribute to a grade of accuracy and confidence in each cybersecurity tool used. One such way to infer an accuracy metric is using Youden's Index that can be seen in equation (1) as OWASP Foundation [5] described for the OWASP Benchmark.

Youden's Index or Youden's J statistic

$$J = TPR(TruePositiveRate) - FPR(FalsePositiveRate) \quad (1)$$

B. SAST - Static Application Security Testing

Expanding on the white-box technique, most vulnerabilities are due to developer's lack of awareness and/or bad programming practices [6] in such a way that OWASP Foundation [7] defines it "as part of a Code Review", a "method of computer program debugging that is done by examining the code without executing the program", and also a good technic to help find:

- "Syntax violations"
- "Security vulnerabilities"
- "Programming errors"
- "Coding standard violations"
- "Undefined values"

1) Symbolic Execution: Inside SAST's set of technics, introduced by [8], we find symbolic execution engines which can help protect systems that range from modern SoC (System-on-Chip) designs and hardware Trojans [9], to binary-level security analysis such as malware comprehension [10] and even a symbiosis between symbolic and concrete execution [11]. All these tools share a common requirement: any instruction set ranging between machine code to source code. These tools excel by mimicking the execution of the analysed software, controlling each value the input influences in the form of symbolic values or variables.

However, a more in-depth look at symbolic execution unveils some obstacles, such as path explosion, constraint solving, environment modelling and others. These symbolic values are allowed to be anything at a first stage, and its value is deciphered further down in the execution path, reducing the possibilities at every branch the execution encounters. The branching happens after performing constraint solving, a very time expensive operation especially when the tested source is of high complexity [12].

Constraint solving occurs at every moment where a variable value impacts the flow of execution or even once a symbolic variable takes part in any arithmetic computation. This escalates once these computations become of a non-linear nature, such as non-linear expressions and non-linear floating-point computations [13].

Path explosion is one concern found in many high performance problems: the input is sufficiently complex for any sort of resource - be it time, memory, or even processing capacity - to escalate tremendously and becoming unreasonable in a standard environment. When it comes to symbolic execution, a new path is encountered every time a constraint is solved; from that point on, that constraint is a path condition. To better understand this concept, here's an example using the tool *KLEE* according to [14, p. 3]:

> "When KLEE detects an error or when a path reaches an exit call, KLEE solves the current path's constraints (called its path condition) to produce a test case that will follow the same path when rerun on an unmodified version of the checked program (e.g., compiled with gcc)."

Environment modelling is another very common problem in software analysis techniques given how much software development relies on third-party libraries today. This problem escalates whenever these dependencies are packaged as binaries, or the sources are 'super-optimized', the paths exploration and consequently the constraint solving may fail which may result in false positives or false negatives [14].

III. SYMBOLIC EXECUTION TOOLS

There are a couple of tools available that perform symbolic execution in numerous ways with different focus and strengths. A good agglomerate of these tools is maintained in a GitHub repository owned by Luckow [15]. There we find symbolic execution tools focused on binaries and various programming languages like Java, JavaScript, or even Rust, but also portable platforms like WASM and LLVM.

This chapter focuses on the tools that were considered for this work, and the reasons behind the final choice (section III-A). Then, the chosen tools, KLEE and Owi, are described in more detail. For both tools, installation instructions,

features, and usage examples are provided (sections III-B and III-C, respectively). Finally, a comparison between both tools is presented in section III-D.

A. Engines Availability

In the endeavour of searching for tools capable of symbolic execution many attempts to have a functional installation of the available alternatives were done, but not all were successfull.

Despite of Cloud9 and Kite having their own websites [16, 17] both did not contain valid hyperlinks for either sources or a compiled binary of the software. On the other hand, for SymJS nothing more than a scientific document was found [18].

That leaves us to KLEE and Owi, a pair of tools with a large support of languages [19, 20], and SPF and Jalangi2, the two programming languages used to develop the WebGoat project maintained by OWASP [21, 22]. OWASP WebGoat project is an intentionally vulnerable application so developers interested in learning cybersecurity can use it to learn about vulnerabilities [22].

Despite the interest around OWASP's WebGoat project and the availability of sources for both SPF and Jalangi2, getting each software to run on their own and consistently is a daunting task with not much documentation to go along and a steep learning curve.

KLEE and Owi are both far better when it comes to the available documentation, and initial learning curve; the users are not bombarded with a codebase and a single 'README' file, instead they have either multiple README's for each subcommand [23] or a large number of papers and websites with tutorials, examples and information on the tool features [14, 24–27].

Given the current software paradigm, where performance is important not only in the matter of software runtime but also when it comes to software development, maintenance, and portability, tools that apply to technologies such as LLVM and WASM are interesting research targets. Therefore, we are looking at KLEE and Owi, which propose solutions for each tech stack, respectively.

B. KLEE

KLEE explores the LLVM infrastructure rather successfully and with speed; however, this engine bottlenecks quite considerably when it is applied on larger scale projects. This is due to its space management, given how it tries to keep all relevant information about the current states. It is also unable to handle dynamically generated code, concurrent execution and struggles with modelling network and peripherals, although most peers share these weaknesses [28]. The LLVM project supports many different languages like Haskell, Rust and Python, but our test cases will focus on C/C++ programs [19].
.

LLVM has been referenced a few times in this document, but a brief explanation is in order. LLVM is not an acronym, but the full name of the project. Despite the name, it has nothing to do with virtual machines. LLVM is "a collection of modular and reusable compiler and toolchain technologies". It is also praised for its versatility, flexibility, and reusability. For more information about LLVM, please refer to their website [19].

[14] describe KLEE as being in both static and dynamic techniques of testing: every symbolic execution engine is static by definition, but KLEE becomes dynamic through its outside environment interaction features.

1) Features: KLEE is packaged with multiple CLIs (see Table II) that contribute to the user experience, but the main command that runs the symbolic execution engine is `klee`.

Command	Description
`kleaver`	Operates on `.kquery` files.
`klee-exec-tree`	Takes `klee` tree output and displays a graphical representation or statistics.
`klee-replay`	Takes one ktest from a file or multiple from a directory and runs them.
`klee-stats`	Takes `klee` stats output and processes it as instructed.
`klee-zesti`	ZESTI[1] like wrapper of KLEE.
`ktest-gen`	Generates test cases (a `.ktest` file) from concrete input.
`ktest-randgen`	Generates test cases (a `.ktest` file) from randomly generated input.
`ktest-tool`	Describes a `.ktest` file containing information on each input object.

[1]ZESTI (Zero-Effort Symbolic Test Improvement)

TABLE II: KLEE commands.

2) Instrumentation: The KLEE web page contains a tutorials section to help get familiarized with the tool. These tutorials contain samples and instructions on how to properly set up a C/C++ source file, let's call it code instrumentation, and how to run and analyse the results. Their sources can be found in higher detail in appendix **??**.

The first, and most basic example, shows how one can mark a variable as symbolic by using the `klee_make_symbolic()` function This function takes 3 arguments; the first two will define a memory range in which the variable information is stored, the third defines a human-readable object name that identifies the symbolic variable in the generated test case files (for the variable '`a`' declared on **??** in **??**, this could be "my variable".):

1) The variable memory address,
2) The variable memory size,
3) The variable name, a human-readable identifier.

Another valuable feature, is the `klee_prefer_cex()` that tells KLEE to prefer a set of values when generating test cases; this allows the user to get readable results like "aaaa" instead of hexadecimal representation of character values such as "\xff\xff\xff\xff".

3) Running KLEE: After instrumenting the source code with KLEE instructions, see section III-B2, the next step is compiling it to LLVM, and the easiest way to achieve that is using Clang (Listing 1).

```
1 clang -emit-llvm -c main.c
```

Listing 1: Basic clang command

When compiling programs that include the klee library, it may be needed to include the klee library, so Clang understands its instructions, which can be done like in Listing 2.

```
clang -I $PATH_TO_KLEE_LIBRARY -emit-llvm -c main
    .c
```

Listing 2: Basic clang command with path to klee library

However, the KLEE documentation recommends a more complete command to have access to a more complete set of functionality that klee provides, such as test replayability, and that command is available in Listing 3.

```
clang -emit-llvm -c -g -O0 -Xclang -disable-O0-
    optnone main.c
```

Listing 3: Complete clang command

The compilation command should return an object with extension '*.bc*' containing the LLVM representation of the program. This file is in the correct format for KLEE, and therefore we can now test it by running the command on Listing 4.

```
klee main.bc
```

Listing 4: Running klee on a program

Running KLEE on a program results in file outputs, the `klee-out-n` and `klee-last` folders are created. For `klee-out-n`, `n` is the zero-indexed counter of the run, and `klee-last` is a symbolic link to the most recent run folder; by the second run of KLEE, `klee-last` points to `klee-out-1`. The folders created by KLEE contain all the generated test cases, error reports, and other useful information about the execution. Each generated test case is stored in a file with extension '*.ktest*' and can be analysed using the `ktest-tool` command (see Listing 5).

```
$ ktest-tool klee-last/test000001.ktest
ktest file : 'klee/klee-last/test000001.ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'my signed integer'
object 0: size: 4
object 0: data: b'\x00\x00\x00\x00'
object 0: hex : 0x00000000
object 0: int : 0
object 0: uint: 0
object 0: text: ....
```

Listing 5: Analyzing a ktest file

C. Owi

[29] describe Owi as a toolkit that aggregates functionality to assist WebAssembly developers, one of which enables the compilation of C code to WASM and running their symbolic interpreter on top of it.

WebAssembly is a portable binary instruction format for executable programs, designed to be a compilation target for high-level languages like C/C++ and Rust, enabling deployment on the web for client and server applications. WebAssembly is designed to be fast, efficient, and secure, making it an ideal choice for running complex applications in web browsers and other environments [30, 31].

1) Features: Owi being a toolchain designed for the development of WASM, it provides a set of subcommands as listed in Table III; however, to be able to use the symbolic interpreter packaged with it, we need to also install a restraint solver. Owi supports any Smt.ml supported solver, but the default solver is Z3.

Owi also supports test replayability from an output file generated by the *sym* subcommand as you can see in Listing 6.

```
owi sym main.wat > main.model
owi replay --replay-file=main.model main.wat
```

Listing 6: Owi replayability model

Subcommand	Description
c	Compiles C source code to WASM and runs the WASM symbolic interpreter on the end result.
conc	Takes WASM input and runs the concolic interpreter on it.
fmt	Formats WebAssembly text format files (*.wat* or *.wast*).
opt	Optimizes WebAssembly modules.
run	Runs the concrete interpreter.
replay	Replay a WASM module containing symbols with concrete values in a replay file containing a model.
script	Runs a reference test suite script.
sym	Takes WASM input and runs the symbolic interpreter on it.
validate	Validates WebAssembly modules.
wasm2wat	Generate a text format file (*.wat*) from a binary format file (*.wasm*).
wat2wasm	Generate a binary format file (*.wasm*) from a text format file (*.wat*).

TABLE III: Owi subcommands

2) Instrumentation: The Owi repository maintains a folder with examples of how to use the different subcommands; similarly to KLEE, we have available multiple examples on how to use `owi c` that are detailed in appendix **??**.

Looking at the first example, some differences from KLEE are evident, specifically how the tool is informed about which variables are symbolic. KLEE uses the variable address and the type is disregarded in the whole process (the generated test cases detail values for each possible primitive type); Owi on the other hand expects to be told the symbolic variable type, through the available functions:

- `owi_i32()` - to define 32-bit integer values, with 8-bit and 64-bit variants;
- `owi_f32()` - to define 32-bit floating-point values, with a 64-bit variant;
- `owi_char()` - to define character values;
- `owi_bool()` - to define boolean values.

Unlike KLEE functions, these do not support any argument that allows defining a custom name for the symbolic variable; the symbols are identified by order of creation - if we define all positions of an array as symbolic, each position symbolic identifier would correspond to their array position, but if any other symbol is defined before the identifiers would shift by the number of predefined variables.

To run Owi on a simple sample we must include the Owi library in the source code so it is possible to find Owi function implementations and generate a valid WebAssembly module. As visible on Listing 7, the variable `x` of type `int` is instrumented using the routine `owi_i32` (line 5) and since this program is trying to find the roots of a polynomial, we want the symbolic values that result in `poly == 0`, therefore line 16 will catch a 'problem' every time `poly` equals 0 since the passed condition evaluates to `false`.

There are in fact solutions to the defined polynomial and Owi outputs them as visible in Listing 8. The output represents a test case, and to replicate the test case one or more of the passed inputs must have a specific set of values. On the example at hand, only one symbol was defined and its state is displayed on line 6. The second value on this line is the symbols' name (`symbol_0`), then follows the symbol's type and since an integer was defined its type `i32`. Finally, its value is also presented, in this case we can state that 4 is one solution to the polynomial written on Listing 7.

Owi also produces a `owi-out/test-suite` folder and inside it a `testcase-N.xml`, where `N` is an index starting on 1, that describes the input value for a symbolic variable for that test case, and a `metadata.xml` that details the arguments of the owi executation such as the source code language, which Owi subcommand produced the file, the input program file ('*poly.c*') and other details on the execution. The generated test cases are relevant to found issues, Owi does not generate test cases on each traced path like KLEE does.

```
1  // this include is required
2  #include <owi.h>
3
4  int main() {
5    int x = owi_i32();
6    int x2 = x * x;
7    int x3 = x * x * x;
8
9    int a = 1;
10   int b = -7;
11   int c = 14;
12   int d = -8;
13
14   int poly = a * x3 + b * x2 + c * x + d;
15
16   owi_assert(poly != 0);
17
18   return 0;
19 }
```

Listing 7: Polynomial example 1 source code with Owi instrumentation

```
1  Assert failure: (bool.ne
2                   (i32.mul
3                    (i32.add (i32.mul (i32.add
       ↪ symbol_0 -7) symbol_0) 14)
4                    symbol_0) 8)
5  model {
6    symbol symbol_0 i32 4
7  }
8  Reached problem!
```

Listing 8: Polynomial example 1 output

3) Running Owi: Similarly to KLEE, Owi provides a set of methods to inform the symbolic interpreter in an attempt to improve the results, examples of this are `owi_assume()` and `owi_i32()` for int32 variables (and others for the respective primitive types such as `owi_char()`).

Once the sources are ready, running the symbolic interpreter is very simple with Owi by running Listing 9. Since this is a toolkit it contains many subcommands, the most relevant for this study being the '*c*' subcommand that allows running C code directly.

```
1  owi c main.c
```

Listing 9: Running owi on a program source.

However, if your program requires any sort of custom option or if you just want to experiment with them, we can 'ask' Owi how they compile the sources. To do that, all one needs to do is run `owi c main.c --debug` on invalid C files and will find in the tool output the command in Listing 10.

```
1  clang -O3 --target=wasm32 -m32 -ffreestanding \
2      ↪ --no-standard-libraries -Wno-everything -
       flto=thin \
3      ↪ -Wl,--entry=main -Wl,--export=main -Wl,--
       lto-O0 \
4      ↪ -Wl,-z,stack-size=8388608 \
5      ↪ -I $HOME/.opam/default/share/owi/libc -o
       main.wasm \
6      ↪ $HOME/.opam/default/share/owi/binc/libc.
       wasm \
7      ↪ main.c
```

Listing 10: Owi abstracted compilation command (clang).

Owi also integrates Frama-C's E-ACSL plugin. This plugin enables the detection of implementation issues, or even custom heuristics for complex bugs, through the definition of a set of rules that can create constraints on acceptable inputs ('requires') and outputs ('ensures'). The `owi c` subcommand and to enable all one needs to do is pass the argument `--e-acsl`. To see how Frama-C is being executed the process is similar as to the compilation, but now with the E-ACSL option, and it looks like Listing 11. This plugin enables the detection of implementation issues, or even custom heuristics for complex bugs, through the definition of a set of rules that can create constraints on acceptable inputs ('requires') and outputs ('ensures').

```
1  frama-c -e-acsl -no-frama-c-stdlib \
2      ↪ -kernel-warn-key CERT:MSC:38=inactive -
       verbose 2 \
3      ↪ -cpp-extra-args="-I$HOME/.opam/default/
       share/owi/libc" \
4      ↪ main.c -then-last -print -ocode main.
       instrumented.c
```

Listing 11: Owi abstracted Frama-C instrumentation of E-ACSL.

After compiling either `main.c` or `main.instrumented.c`, the resulting `.wasm` object can be tested with Owi using the command on Listing 12.

```
owi sym main.wasm
```

Listing 12: Running owi on a compiled program.

Executing either `c` or `sym` subcommands generates some files inside the folder 'owi-out' created on the current working directory: `owi sym` only generates test case files named 'testcase-n.xml' being n the number of results detected and starting from 1. The subcommand `owi c` also generates a file describing the test cases that fail execution, but also outputs an additional 'metadata.xml' , and a 'a.out.wasm' file (the compiled sources); however, the test case and metadata files are generated in a subfolder called 'test-suite' while the WebAssembly module is created on the current working directory (at the same level as 'owi-out' folder). For a visual reference of this file structure take a look at Listing 13, consider 'main.c' the input sources and 'main.wasm' pre-compiled before running Owi.

```
# owi c ...
.
├── a.out.wasm
├── main.c
└── owi-out
    └── test-suite
        ├── metadata.xml
        └── testcase-1.xml

# owi sym ...
.
├── main.c
├── main.wasm
└── owi-out
    └── testcase-1.xml
```

Listing 13: Owi generated files structure.

D. Feature Comparison

Not only it is important to understand how well a tool performs, but also how flexible it can be in the different scenarios it may face. Table IV was built describing each tool supported vulnerabilities, their ease-of-use, how easy they are to install, and which STP solvers they support. The tools' installation speed and complexity are scaled from 1-5 ranging from very slow to very fast or very hard to very easy, respectively. The third and fourth columns (Ease-of-use for beginner and experienced developers) take the same scale as the second one: 1-5, very hard to very easy.

The tools' installation process is very different both in the amount of methods but also in the complexity of the provided solutions. Although the recomended solution by The KLEE Team [27] is very complex since it requires a lot of dependencies and different processes, there is also available a quick start option using a docker image made available by the maintainers of KLEE. In contrast, Owi does not offer many ways to install its engine but both of them require the installation of `opam` but also `dune` if one compiles the source code directly.

With their differences in mind, KLEE punctuates a score of 5 in installation speed but if we disregard the Docker approach, both KLEE and Owi take 2 points; even though the first requires much more commands and dependencies, the second takes more time to actually compile the repository.

When it comes to installation complexity, KLEE clearly could use some improvement in build automatization while Owi leverages this quite well with `opam` and `dune` however these are tools very tied with OCaml development which is not at all the focus of this study adding unnecessary complexity to the installation process. Considering this, KLEE scores a 1 in the recommended process. But if using Docker is an option, then it scores a 4. Owi on the other hand scores a 3, much because of the OCaml tools; we're intentionally ignoring the opam package since it did not produce a valid installation, otherwise Owi could score a 5.

The tools usability is also scored in the perspective of both beginners and experienced developers. Because KLEE has so much documentation available across the internet the learning curve is very stable but the more advanced the requirements the more complex the usage may get due to how some functionalities are scattered across different CLI's, these two points contribute for a solid positive score (4). Owi on the other hand can pose as a hard obstacle for the least experienced mainly due to the lack of support documents and the fact that OCaml doesn't make it into the top 50 of most popular programming languages where the 50th most popular language rates 0.15%, according to TIOBE[32], scoring 2 points in this scenario. The popularity issues become nearly non-existent when an experienced mind makes use of the tool and the simplicity of having all tools aggregated into one CLI also contributes to a better score (4).

The second section of Table IV describes features and heuristics supported by each tool; heuristics refers to software vulnerabilities/errors, if they are supported by the tools and how these heuristics are defined. This is analysed in higher detail in section IV.

The last section of Table IV aggregates information from KLEE and Owi documentation on SAT and SMT solvers. While SAT solvers handle *boolean satisfiability*, SMT expand on this concept into general formulas[33]. Examples of SMT solvers are STP and Z3 but there are also wrappers like MetaSMT and frontends as Smt.ml [34–37]. MiniSat is a SAT solver used by STP as the underlying SAT solver and Smt.ml also has MiniSat in its support roadmap [37, 38].

IV. EXPERIENCES AND RESULTS

In this chapter, multiple experiences using both KLEE and Owi tools will be described and analysed. We will start by describing the test environment in section IV-A, and then proceed to describe each experience in its own section. Finally, a summary of all results and an overview of the experiences will be presented in section IV-F.

Each experience will showcase their capabilities and limitations.

ADALogics [39] describe themselves as "a world-class player in software security" and as such they have many interactions in software security applications, such as symbolic

*F*eature	KLEE	Owi
Installation speed	5 (2)	2
Installation complexity	1 (4)	3
Ease-of-use (begginer)	4	2
Ease-of-use (experienced)	4	4
Heuristics support	Larger	Reduced
Heuristics definition	Hardcoded	Hardcoded
Supports E-ACSL	NO	YES
Supports MetaSMT	YES	NO
Supports STP	YES	NO
Supports Z3	YES	YES
Supports metaSMT solvers	YES	NO
Supports Smt.ml solvers	NO	YES
Supports MiniSat	YES	NO

TABLE IV: Feature comparison

execution. One of these interactions is a blog on KLEE where they go through the installation and usage of the tool to find bugs. On this blog, ADALogics [24] presents 4 videos to demonstrate KLEE set of features and in video 3 they discuss vulnerabilities of the types:

- Global Buffer Overflow (Sample 6 - Global Buffer Overflow),
- Stack-based Buffer Overflow (Sample 7 - Stack-based Buffer Overflow),
- NULL pointer dereference (Sample 8 - NULL pointer dereference),
- Use after `free` (Sample 9 - Use after free).

Sections IV-B to IV-E will cover these vulnerabilities using both KLEE and Owi by instrumenting each source in the intended way.

A. Test Environment

To increase the results' credibility, all tests were run on the same machine, therefore sharing the same resources. The specifications of such machine consisted in a 12th Gen Intel Core processor (i7-1255U) of 10 cores (and a total of 12 threads) with a max clock speed of 4.70 GHz, and 20 GB (4 + 16) of DDR4 3200 MHz RAM.

This is the parent system setup with a Windows 11 installation; however, the test environment was created in a WSL environment within this machine which shares 4 cores and 8 GB of RAM available.

For simplicity of testing, all samples were organized by folders and subfolders: the parent folder aggregates by sample (1, 2, 3, etc.) and child folders aggregates files by tool, KLEE or Owi; a folder description is visible in Listing 14.

Each child folder (e.g., sample1/klee, sample1/owi) contains its own 'main.c' file with the sample source code and will also contain any generated output by the tool. On KLEE runs, the `klee-out-n` (where `n` is the number of the run) and `klee-last` folders are created, and on Owi, the `owi-out`. There are also the compiled objects `main.bc` and `main.wasm` files for targets LLVM and WASM platforms, respectively.

```
.
├── sample1
│   ├── klee
│   └── owi
├── sample2
│   ├── klee
│   └── owi
...
└── sample9
    ├── klee
    └── owi
```

Listing 14: Samples folder structure (samples 1-5)

```
./sample1
├── klee
│   ├── klee-last -> klee-out-0
│   ├── klee-out-0
│   ├── klee-output.txt
│   ├── klee-time.txt
│   ├── main.bc
│   └── main.c
└── owi
    ├── main.c
    ├── main.wasm
    ├── owi-out
    ├── owi-output.txt
    └── owi-time.txt
```

Listing 15: Sample file structure

B. Sample 6 - Global Buffer Overflow

On this section, we will be covering an example of a Global Buffer Overflow vulnerability and how these problems can be detected using KLEE and Owi.

On **??**, we can see how the `test1` input variable `arr` is safely marked as symbolic with **????**, using the KLEE instrumentation methods. **??** makes the array safe for string operations like '`strcmp`' on **??**.

The second instrumentalization is done on Listing 16. Because the symbolic input is an array, each position must be initialized (see lines 19 to 22) and because this is an array of character values (a string) we're telling Owi the string value is suffixed by '\0' using `owi_assume` function and passing the condition `arr[9] == '\0'` as its argument (line 23). The string is 'closed' so that it is safe for string operations, such as '`strcmp`' on line 10, and it isn't reported as a problem.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <owi.h>
5
6  int global_arr[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8,
       ↪ 9};
7  int test1(char *buf)
8  {
9    int i = 5;
10   if (strcmp(buf, "BUG!") == 0)
11   {
12     i += 20;
13   }
14   return global_arr[i];
15 }
16
17 int main(int argc, char const *argv[])
18 {
19   char arr [10];
20   for (int i = 0; i < 10; i++) {
21     arr[i] = owi_char();
22   }
```

```
23   owi_assume(arr[9] == '\0');
24
25   test1(arr);
26
27   return 0;
28 }
```

Listing 16: Sample 6 instrumented for Owi

Starting with klee execution, the results are found very quickly and only 6 paths are traced, one of which is partial and an error detection. Listing 17 also shows on line 1 the type of error ("`memory error: out of bound pointer`") and on which file and line the error occurred ("`main.c:14`"). All unique errors generate a test case of its own and the one detected for this sample is expressed in detail on Listing 18.

On the other hand, owi displays "`All OK`" after scanning the sources stating it did not find the vulnerability.

```
1 KLEE: ERROR: main.c:14: memory error: out of
      ↪ bound pointer
2 KLEE: NOTE: now ignoring this error at this
      ↪ location
3
4 KLEE: done: total instructions = 12493
5 KLEE: done: completed paths = 5
6 KLEE: done: partially completed paths = 1
7 KLEE: done: generated tests = 6
```

Listing 17: Sample 6 - KLEE output

```
1 ktest file : './sample6/klee/klee-last/test000003
      ↪ .ktest'
2 args       : ['main.bc']
3 num objects: 1
4 object 0: name: 'arr'
5 object 0: size: 10
6 object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
7 object 0: hex : 0x4255472100ffffffff00
8 object 0: text: BUG!......
```

Listing 18: Sample 6 - KLEE test case for the detected error

C. Sample 7 - Stack-based Buffer Overflow

Differently from Sample 6 - Global Buffer Overflow, Stack-based Buffer Overflow vulnerabilities look at scoped and dynamically allocated variables like the ones on line 8 in both KLEE and Owi samples (**??** and listing 21); the samples' instrumentation is identical to what is seen on Sample 6 - Global Buffer Overflow.

Running KLEE results in a very similar result to that seen on section IV-B, once again 6 generated tests, one of which triggered an execution error shown in detail on Listing 20. This time, the error occurs on **??** of file main (**??**). Owi fails to find the vulnerability, again.

```
1 KLEE: NOTE: KLEE: ERROR: main.c:26: memory error:
      ↪  out of bound pointer
2 now ignoring this error at this location
3
4 KLEE: done: total instructions = 12566
5 KLEE: done: completed paths = 5
6 KLEE: done: partially completed paths = 1
7 KLEE: done: generated tests = 6
```

Listing 19: Sample 7 - KLEE output

```
1 ktest file : './sample7/klee/klee-last/test000003
      ↪ .ktest'
2 args       : ['main.bc']
3 num objects: 1
4 object 0: name: 'arr'
5 object 0: size: 10
6 object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
7 object 0: hex : 0x4255472100ffffffff00
8 object 0: text: BUG!......
```

Listing 20: Sample 7 - KLEE test case for the detected error

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <owi.h>
5
6  int test2(char *buf)
7  {
8    int *dyn_mem_arr = malloc(sizeof(int) * 10);
9    dyn_mem_arr[0] = 0;
10   dyn_mem_arr[1] = 1;
11   dyn_mem_arr[2] = 2;
12   dyn_mem_arr[3] = 3;
13   dyn_mem_arr[4] = 4;
14   dyn_mem_arr[5] = 5;
15   dyn_mem_arr[6] = 6;
16   dyn_mem_arr[7] = 7;
17   dyn_mem_arr[8] = 8;
18   dyn_mem_arr[9] = 9;
19
20   int i = 5;
21   if (strcmp(buf, "BUG!") == 0)
22   {
23     i += 20;
24   }
25
26   int return_value = dyn_mem_arr[i];
27   free(dyn_mem_arr);
28   return return_value;
29 }
30
31 int main(int argc, char const *argv[])
32 {
33   char arr[10];
34   for (int i = 0; i < 10; i++)
35   {
36     arr[i] = owi_char();
37   }
38   owi_assume(arr[9] == '\0');
39
40   test2(arr);
41   return 0;
42 }
```

Listing 21: Sample 7 - Owi instrumentation

D. Sample 8 - NULL pointer dereference

To test NULL pointer dereference bugs, a small string array ('`char *global_string[]`' on line 6) is initialized and used in function '`test3`' where on line 21 will trigger an error when the input is favorable.

After the samples are instrumented, the tools are run and a pattern begins to show where KLEE detects the vulnerability but Owi does not. See **??** and listing 22 for KLEE and Owi instrumented samples, respectively, Listing 23 for KLEE terminal output and Listing 24 for the generated test case.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <owi.h>

char *global_string[] = {
  "string1",
  "string2",
  "string3",
  NULL,
};

int test3(char *buf)
{
  int i = 1;
  if (strcmp(buf, "BUG!") == 0)
  {
    i = 3;
  }

  char c = *(global_string[i]);
  if (c == 's')
    return 0;
  return 1;
}

int main(int argc, char const *argv[])
{
  char arr[10];
  for (int i = 0; i < 10; i++) {
    arr[i] = owi_char();
  }
  owi_assume(arr[9] == '\0');

  test3(arr);
  return 0;
}
```

Listing 22: Sample 8 - Owi instrumentation

```
KLEE: ERROR: main.c:21: memory error: out of
    ↪ bound pointer
KLEE: NOTE: now ignoring this error at this
    ↪ location

KLEE: done: total instructions = 12549
KLEE: done: completed paths = 5
KLEE: done: partially completed paths = 1
KLEE: done: generated tests = 6
```

Listing 23: Sample 8 - KLEE output

```
ktest file : './sample8/klee/klee-last/test000003
    ↪ .ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'arr'
object 0: size: 10
object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
object 0: hex : 0x4255472100ffffffff00
object 0: text: BUG!......
```

Listing 24: Sample 8 - KLEE test case for the detected error

E. Sample 9 - Use after free

Finally, 'Use after free' runtime errors can also become very hard to detect by the human eye when code complexity increases; '`test4`' function presents a simple case of this vulnerability to showcase KLEE and Owi capabilities. **??** and listing 28 illustrate instrumented sources for each symbolic execution engine.

KLEE detected the bug on **??**, generated 6 tests, and distinguished the error from the ones on sections IV-B to IV-D. KLEE's output (Listing 25) shows that the memory error is of type '`double free`' on line 1, and the generated test case (Listing 26) shows the expected input value.

```
KLEE: ERROR: main.c:14: memory error: double free
KLEE: NOTE: now ignoring this error at this
    ↪ location

KLEE: done: total instructions = 12484
KLEE: done: completed paths = 5
KLEE: done: partially completed paths = 1
KLEE: done: generated tests = 6
```

Listing 25: Sample 9 - KLEE output

```
ktest file : './sample9/klee/klee-last/test000003
    ↪ .ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'arr'
object 0: size: 10
object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
object 0: hex : 0x4255472100ffffffff00
object 0: text: BUG!......
```

Listing 26: Sample 9 - KLEE test case for the detected error

While KLEE detected the double free as any other code issue, Owi reported the problem as an internal error and its respective stack trace as displayed in Listing 27. This behaviour suggests the tool does not properly support this issue and might cause the scan to miss other issues because of this internal error.

```
owi: internal error, uncaught exception:
     Failure("Memory leak double free")
     Raised at Stdlib__Domain.join in file "
    ↪ domain.ml", line 296, characters 16-24
     Called from Stdlib__Array.iter in file "
    ↪ array.ml", line 113, characters 31-48
     Called from Owi__Wq.read_as_seq.(fun) in
    ↪ file "src/data_structures/wq.ml", line 17,
    ↪  characters 4-16
     Called from Owi__Cmd_sym.cmd.
    ↪ print_and_count_failures in file "src/cmd/
    ↪ cmd_sym.ml", line 99, characters 10-20
     Called from Owi__Cmd_sym.cmd in file "src/
    ↪ cmd/cmd_sym.ml", line 130, characters
    ↪ 15-49
     Called from Cmdliner_term.app.(fun) in file
    ↪ "cmdliner_term.ml", line 24, characters
    ↪ 19-24
     Called from Cmdliner_eval.run_parser in file
    ↪  "cmdliner_eval.ml", line 35, characters
    ↪ 37-44
```

Listing 27: Sample 9 - Owi output

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <owi.h>

int test4(char *buf)
{
```

```
8    char *var = malloc(10);
9    int i = 5;
10   if (strcmp(buf, "BUG!") == 0)
11   {
12     free(var);
13   }
14   free(var);
15
16   return 1;
17 }
18
19 int main(int argc, char const *argv[])
20 {
21   char arr[10];
22   for (int i = 0; i < 10; i++) {
23     arr[i] = owi_char();
24   }
25   owi_assume(arr[9] == '\0');
26
27   test4(arr);
28   return 0;
29 }
```

Listing 28: Sample 9 - Owi instrumentation

F. Results

Both KLEE and Owi require compiled sources, however Owi can abstract that process for the user if they are testing C source code. To avoid our statistics to include resources spent in source compilation, we will compile our sources for Owi using the same command they do in their abstraction. Please note that the comparisons in Table IV - Feature comparison take only symbolic execution features into consideration.

This section discusses **Accuracy** and **Performance** metrics such as:

Accuracy Metrics

- **TP - True Positives**
  Number of **expected** vulnerabilities **reported**.
- **TN - True Negatives**
  Number of **unexpected** vulnerabilities **not reported**.
- **FP - False Positives**
  Number of **unexpected** vulnerabilities **reported**.
- **FN - False Negatives**
  Number of **expected** vulnerabilities **not reported**.
- **Youden Index or Youden's J statistic**
  The accuracy metric used in OWASP Benchmark [40] is the following
  $J = TPR - FPR$

$$= (\frac{TP}{TP+FN})^{a} - (\frac{FP}{TN+FP})^{b}$$

$$= \frac{TP*TN - FP*FN}{(TP+FN)(TN+FP)}$$

[a] TPR - True Positive Rate
[b] FPR - False Positive Rate

Performance Metrics

- **Paths traced**
  Branches of execution created to navigate.
- **CPU (%)**
  The CPU used by the process, in percentage.
- **Memory (KB)**
  The 'maximum resident set size' (very similar to peak memory) used by the process, in kilobytes.
- **Time (s)**
  The execution time of the process, in seconds.

Accuracy metrics were extracted by manually inspecting each scan output, and performance metrics such as CPU, memory and time of execution were measured using the GNU `time` program (see Listing 29). "**Paths traced**" was also extracted manually similarly to accuracy metrics. There is also the "**Success**" metric that indicates whether the sample was successfully analysed by the test tool, without regard for results.

```
1 # to avoid confusion with bash keyword 'time'
2 # use the full path given to you by `which time`
3
4 time -v klee main.bc
5
6 time -v owi sym main.wasm
```

Listing 29: Extraction of performance metrics

After executing both engines on all test samples and extracting the results from each of them, Tables V and VI we're created. With these metrics properly formatted, calculating the "**Youden Index**" for both tools is easier, higher is better.

It is apparent how KLEE detected much more vulnerabilities than Owi, which plays in its favour, and the calculations on equations (2) and (3) reflect this statement, KLEE scores approximately 54% while Owi stands on 29% of bug detection accuracy.

If we look at the Youden Index interpretation guide from OWASP Benchmark [40] shown in Figure 1, we can see that KLEE scores highly (87.5%) in the *y* axis (TPR), while Owi scores poorly (28.5%). Both tools score low on the *x* axis (FPR), with KLEE at 33.3% and Owi at 0%. This means that KLEE is very good at detecting vulnerabilities, but also has a higher rate of false positives, while Owi is not very good at detecting vulnerabilities, but has a low rate of false positives. A larger set of samples would be needed to draw more concrete conclusions.

Sample	Success	TP	TN	FP	FN
1	YES	0	1	0	0
2	YES	2	0	0	0
3	YES	1	0	0	0
4	YES	0	1	0	0
5	YES[1]	0	0	1	1
6	YES	1	0	0	0
7	YES	1	0	0	0
8	YES	1	0	0	0
9	YES	1	0	0	0

TABLE V: KLEE accuracy metrics

By running KLEE and Owi on the same set of samples, some differences are visible in their outputs and in their performances (see Section IV-F). KLEE was overall slower, with the only exception being the primes example, but was

Sample	Success	TP	TN	FP	FN
1	YES	0	1	0	0
2	YES	0	0	0	1
3	YES	1	0	0	0
4	YES	0	1	0	0
5	YES	1	0	0	0
6	YES	0	0	0	1
7	YES	0	0	0	1
8	YES	0	0	0	1
9	YES	0	0	0	1

TABLE VI: Owi accuracy metrics

$$J = (\frac{7}{7+1})^{*} - (\frac{1}{2+1})^{**} \approx (0.875) - (0.333) \approx 0.54 \quad (2)$$

KLEE Youden Index

$$J = (\frac{2}{2+5})^{*} - (\frac{0}{2+0})^{**} \approx (0.285) - (0) \approx 0.29 \quad (3)$$

Owi Youden Index

* TPR - True Positive Rate

** FPR - False Positive Rate

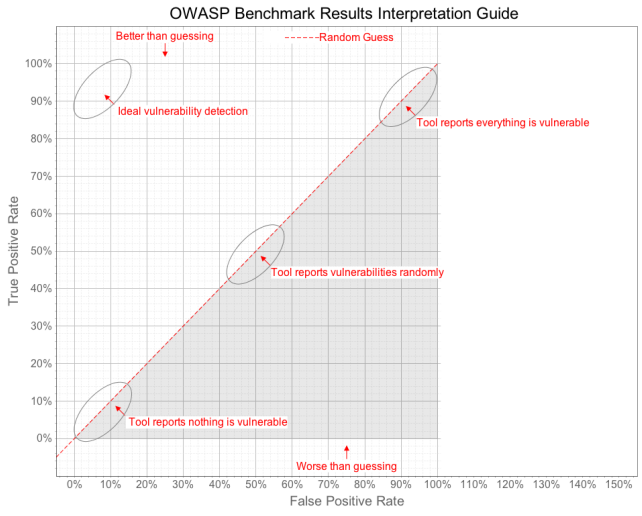

Fig. 1: OWASP Benchmark Scoring Guide - Youden Index interpretation [40]

also the example that consumed less CPU but also Memory on all samples.

Unfortunately, Owi does not seem to output information regarding the travelled states, even after looking at debug logs (which depict execution instructions, but nothing was found to support a number of traversed paths).

An example of how the performance between these tools differ is with sample 3, a test where KLEE traces 306 paths. KLEE reported results consuming nearly half the CPU resources and approximately 73% of the memory usage and 35% of the time spent, that Owi required to run and return no results on the sample.

Sample 2 shows an even greater difference; KLEE spends nearly half the CPU and 19% of the memory usage when compared to Owi, and does that in 1% of the time it took Owi to analyse the same source. This scenario might be a clear indicator of Owi limitations.

#[a]	Success	Paths traced	CPU (%)	Memory (KB)	Time (s)[b]
1	YES	3	99	98'156	0.29
2	YES	6'675	101	139'680	12.93
3	YES	305	93	99'152	0.72
4	YES	99	98	99'504	2.93
5	YES	1	81	98'036	0.39
6	YES	5	102	98'920	0.30
7	YES	5	87	97'536	0.37
8	YES	5	96	98'248	0.33
9	YES	5	96	99'236	0.32

TABLE VII: KLEE performance metrics

[a]Sample number
[b]The best out of five runs

#[a]	Success	Paths traced	CPU (%)	Memory (KB)	Time (s)[b]
1	YES	N/A	108	110'000	0.05
2	YES	N/A	200	725'892	901.51
3	YES	N/A	199	134'928	2.10
4	YES	N/A	204	145'676	9.90
5	YES	N/A	205	151'800	11.13
6	YES	N/A	116	120'172	0.06
7	YES	N/A	115	119'172	0.06
8	YES	N/A	118	118'220	0.05
9	YES	N/A	123	121'040	0.05

TABLE VIII: Owi performance metrics

[a]Sample number
[b]The best out of five runs

REFERENCES

[1] R. Croft, D. Newlands, Z. Chen, and A. M. Babar, "An empirical study of rule-based and learning-based approaches for static application security testing," *International Symposium on Empirical Software Engineering and Measurement*, 10 2021. [Online]. Available: http://eprints.whiterose.ac.uk/95628/

[2] M. Sharma, "Review of the benefits of dast (dynamic application security testing) versus sast," *International Journal of Management and Engineering Research*, vol. 1, pp. 1–4, 6 2021. [Online]. Available: http://www.ijmer.org/index.php/journal/article/view/2

[3] A. Hoffman, *Web Application Security*. O'Reilly Media, Inc., 2024.

[4] L. Dencheva, "Comparative analysis of static application security testing (sast) and dynamic application security testing (dast) by using open-source web application penetration," Master's thesis, National College of Ireland, 8 2022. [Online]. Available: https://norma.ncirl.ie/id/eprint/5956

[5] OWASP Foundation. (2024) OWASP Benchmark — Scoring. [Online]. Available: https://owasp.org/www-project-benchmark/#div-scoring

[6] M. Alenezi and Y. Javed, "Open source web application security: A static analysis approach," *IEEE*, pp. 1–5, 2016.

[7] OWASP Foundation. (2023) OWASP DevSecOps Guideline - v-0.2 — 2-a - Static Application Security Testing. [Online]. Available: https://owasp.org/www-project-devsecops-guideline/latest/02a-Static-Application-Security-Testing

[8] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, pp. 385–394, 7 1976.

[9] A. Vafaei, N. Hooten, M. Tehranipoor, and F. Farahmandi, "Symba: Symbolic execution at c-level for hardware trojan activation," *Proceedings - International Test Conference*, pp. 223–232, 2021.

[10] R. David, S. Bardin, T. D. Ta, J. Feist, L. Mounier, M. L. Potet, and J. Y. Marion, "Binsec/se: A dynamic symbolic execution toolkit for binary-level analysis," *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering, SANER 2016*, vol. 1, pp. 653–656, 5 2016.

[11] F. Gritti, L. Fontana, E. Gustafson, F. Pagani, A. Continella, C. Kruegel, and G. Vigna, "Symbion: Interleaving symbolic with concrete execution," *2020 IEEE Conference on Communications and Network Security, CNS 2020*, 6 2020.

[12] G. Zhang, Z. Chen, Z. Shuai, Y. Zhang, and J. Wang, "Synergizing symbolic execution and fuzzing by function-level selective symbolization," *Proceedings - Asia-Pacific Software Engineering Conference, APSEC*, vol. 2022-December, pp. 328–337, 12 2022.

[13] D. Kroening and O. Strichman, *Decision Procedures - An Algorithmic Point of View*, 1st ed. Springer Berlin, Heidelberg, 4 2008.

[14] C. Cadar, D. Dunbar, and D. Engler, "Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs," *KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs*, 2008. [Online]. Available: https://purl.stanford.edu/fx501rc2898

[15] K. Luckow. (2017, 7) ksluckow/awesome-symbolic-execution. A curated list of awesome symbolic execution resources including essential research papers, lectures, videos, and tools. [Online]. Available: https://github.com/ksluckow/awesome-symbolic-execution

[16] Dependable Systems Lab. Cloud9 - Automated Software Testing at Scale. [Online]. Available: https://dslab.epfl.ch/research/cloud9/

[17] C. G. do Val. Kite: A conflict-driven symbolic execution tool. [Online]. Available: https://www.cs.ubc.ca/labs/isd/Projects/Kite/

[18] G. Li, E. Andreasen, and I. Ghosh, "SymJS: automatic symbolic testing of JavaScript web applications," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. New York, NY, USA: ACM, Nov. 2014.

[19] LLVM Project. The LLVM Compiler Infrastructure Project. [Online]. Available: https://llvm.org/

[20] Fermyon. WebAssembly Language Support Matrix — Fermyon Developer. [Online]. Available: https://developer.fermyon.com/wasm-languages/webassembly-language-support

[21] OWASP Foundation. WebGoat/WebGoat: WebGoat is a deliberately insecure application. [Online]. Available: https://github.com/WebGoat/WebGoat

[22] ——. OWASP WebGoat. [Online]. Available: https://owasp.org/www-project-webgoat/

[23] Formal Security for Web Technologies. OCamlPro/owi: WebAssembly Swissknife & cross-language bugfinder. [Online]. Available: https://github.com/OCamlPro/owi

[24] ADALogics. Symbolic execution with KLEE: From installation and introduction to bug-finding in open source software. [Online]. Available: https://adalogics.com/blog/symbolic-execution-with-klee

[25] The KLEE Team. Tutorial One · KLEE. [Online]. Available: https://klee-se.org/tutorials/testing-function

[26] ——. Tutorial Two · KLEE. [Online]. Available: https://klee-se.org/tutorials/testing-regex

[27] ——. Building KLEE with LLVM 13. [Online]. Available: https://klee-se.org/build/build-llvm13/

[28] N. M. d. Sabino, "Automatic vulnerability detection: Using compressed execution traces to guide symbolic execution," Master's thesis, Instituto Superior Técnico, 2019.

[29] L. Andrès, F. Marques, A. Carcano, P. Chambart, J. F. F. dos Santos, and J.-C. Filliâtre, "Owi: Performant parallel symbolic execution made easy, an application to webassembly," *The Art, Science, and Engineering of Programming*, vol. 9, 10 2024. [Online]. Available: https://hal.science/hal-04627413

[30] WebAssembly. WebAssembly. [Online]. Available: https://webassembly.org

[31] ——. Use Cases - WebAssembly. [Online]. Available: https://webassembly.org/docs/use-cases/

[32] TIOBE. TIOBE Index - TIOBE. [Online]. Available: https://www.tiobe.com/tiobe-index/

[33] RBC Borealis. Tutorial #9: SAT Solvers I: Introduction and applications. [Online]. Available: https://rbcborealis.com/research-blogs/tutorial-9-sat-solvers-i-introduction-and-applications/

[34] The STP Project. STP • The Simple Theorem Prover.

[Online]. Available: https://stp.github.io/
[35] Microsoft Corporation. Documentation for Online Z3 Guide — Online Z3 Guide. [Online]. Available: https://microsoft.github.io/z3guide/
[36] Group of Computer Architecture (AGRA). agra-uni-bremen/metaSMT. [Online]. Available: https://github.com/agra-uni-bremen/metaSMT
[37] Formal Security for Web Technologies. formalsec/smtml: A frontend for multiple SMT solvers in OCaml. [Online]. Available: https://github.com/formalsec/smtml
[38] N. Eén and S. Niklas. MiniSat Page. [Online]. Available: http://minisat.se/
[39] ADALogics. About us — ADA Logics. [Online]. Available: https://adalogics.com/about-us
[40] OWASP Foundation. (2023) OWASP Benchmark — OWASP Foundation. [Online]. Available: https://owasp.org/www-project-benchmark/